



Reconocimiento de Elementos en el Desarrollo de un Programa Informático

Faisal Baroudi Tarfa



Índice

1. Análisis teórico de conceptos	3
1.1. Explicación de conceptos básicos	3
1.2. Clasificación de lenguajes de programación	4
2. Actividad práctica y de análisis	5
2.1. Identificación de paradigmas de programación a partir de ejemplos	5
2.2. Imperativo VS Declarativo	7
2.2.1. Descripción imperativa	7
2.2.2. Descripción declarativa	7



1. Análisis teórico de conceptos

1.1. Explicación de conceptos básicos

- **Código fuente:** Es el texto o conjunto de instrucciones que escribimos usando un lenguaje de programación. Es la versión del programa que las personas podemos leer y entender, pero que el procesador no puede entender directamente.
 - **Código objeto:** Es el resultado que se obtiene al terminar el proceso de traducción del código fuente. Aunque ya esté en un formato que la máquina puede procesar, todavía no está completo ni listo para funcionar por sí solo.
 - **Código ejecutable:** Es el archivo final que enlaza el código objeto con las librerías necesarias del sistema. Este es el único código que realmente puede entrar en el procesador para ser ejecutado y hacer que el programa funcione.
-
1. **Análisis léxico:** Esta fase comprueba que todos los elementos y palabras que has escrito pertenecen realmente al lenguaje de programación que estás usando.
Ejemplo: Si el lenguaje usa la palabra reservada "mostrar" y tú escribes "mostrr", esta fase detecta que la palabra no existe en su vocabulario.
 2. **Análisis sintáctico:** Permite determinar si las palabras y elementos analizados en el paso anterior están colocados en el orden correcto según las reglas del lenguaje.
Ejemplo: Si el lenguaje exige que toda instrucción termine con un ";", esta fase dará un error si olvidamos ponerlo al final de una línea.
 3. **Análisis semántico:** Se encarga de comprobar si las sentencias que has escrito tienen un sentido lógico.
Ejemplo: Si intentamos multiplicar una palabra de texto por un número, la gramática puede estar bien, pero esta fase detectará que la operación matemática no tiene sentido.
 4. **Generación de código intermedio:** Es una fase opcional en la que, una vez descartados los errores, se crea una representación del programa que es independiente del procesador en el que se va a ejecutar.
Ejemplo: Generar un código "neutro" en el que más adelante servirá tanto para adaptar el programa a Windows como a Mac.



5. **Optimización de código intermedio:** También es opcional y su objetivo es intentar mejorar el código generado para que el programa tenga un mejor rendimiento.
Ejemplo: El sistema detecta variables que se han creado pero que nunca se usan y las elimina para ahorrar memoria.
6. **Generación de código final:** En esta última fase se genera el código objeto definitivo, el cual ya dependerá del conjunto de instrucciones específicas de la CPU que vayas a utilizar.
Ejemplo: La traducción final de tu programa a los ceros y unos exactos que entiende el procesador del ordenador.

1.2. Clasificación de lenguajes de programación

- **Según su nivel de abstracción:** El nivel de abstracción mide la capacidad que tiene el lenguaje para ocultar los detalles técnicos al programador.
1. **Nivel Alto:** Son lenguajes que se acercan al lenguaje natural humano y nos ocultan los problemas de depender de una plataforma concreta.
Ejemplo: Java y Kotlin. Pertenecen a esta categoría porque permiten crear aplicaciones complejas sin tener que preocuparse de cómo funciona el hardware interno del dispositivo.
 2. **Nivel Medio:** Son lenguajes que antiguamente se consideraban de alto nivel, pero que con la evolución de la tecnología se han quedado en un punto intermedio.
Ejemplo: C y Fortran. Pertenecen a esta categoría porque combinan una sintaxis comprensible para las personas con la capacidad de controlar el hardware de forma más directa, logrando un mayor rendimiento.
 3. **Nivel Bajo:** Son los lenguajes que dependen completamente de las características técnicas y arquitectónicas de cada máquina.
Ejemplo: Lenguaje Ensamblador y Código Máquina. Pertenecen a esta clasificación porque tienen el nivel de abstracción más bajo, obligando al programador a comunicarse usando las instrucciones exactas del procesador.



- Según su paradigma de programación:
 1. **Imperativo:** Permiten crear algoritmos indicando a través de instrucciones y expresiones los pasos exactos que debe seguir el programa. Le decimos al ordenador cómo hacer las cosas paso a paso.
Ejemplo: C++ y Swift. Pertenecen a este paradigma porque están diseñados para detallar secuencias de acciones, siendo ideales para programar cosas como motores gráficos de videojuegos donde se controla cada aspecto del proceso.
 2. **Declarativo:** Especifican únicamente los valores o resultados que se esperan obtener al final, utilizando relaciones lógicas o matemáticas, sin detallar las instrucciones exactas para llegar a ellos. Le dices al ordenador qué quieres, y el sistema decide cómo hacerlo.
Ejemplos: Haskell y SQL. Pertenecen a esta categoría porque el código se centra en la lógica del problema. Haskell es muy utilizado para motores de Inteligencia Artificial, y SQL se usa en bases de datos para pedir información concreta sin programar el recorrido interno de búsqueda.

2. Actividad práctica y de análisis

2.1. Identificación de paradigmas de programación a partir de ejemplos

- **Fragmento 1 (Sumar números uno por uno):**
 - *Paradigma:* Imperativo
 - *Justificación:* Los lenguajes imperativos se basan en crear algoritmos a través de un conjunto de instrucciones y expresiones. Cuando describimos que el programa debe recorrer la lista y sumar los números, le estamos dando a la máquina las instrucciones exactas paso a paso para resolver el problema.



- **Fragmento 2 (Buscar empleados mayores de 30):**
 - *Paradigma:* Declarativo
 - *Justificación:* La característica principal de los lenguajes declarativos es que solo especifican los valores que se esperan obtener al final, sin detallar qué hay que hacer para lograr ese resultado. En este caso pedimos los nombres de los empleados que sean mayores de 30, sin decirle a la base de datos como tiene que buscarlos.

- **Fragmento 3 (Calcular el factorial matemáticamente):**
 - *Paradigma:* Declarativo
 - *Justificación:* En el temario indica que, en el paradigma declarativo, es necesario indicar las relaciones lógicas y matemáticas para poder llegar al resultado. En este fragmento, el programa no da instrucciones sobre cómo calcular, sino que te dice la regla matemática factorial para que el programa sepa el resultado.

- **Fragmento 4 (Filtrar productos recorriendo la lista uno por uno):**
 - *Paradigma:* Imperativo
 - *Justificación:* Estamos definiendo el algoritmo indicando el conjunto de instrucciones que debe seguir el lenguaje. Al programa le estamos explicando cómo tiene que trabajar exactamente, en vez de pedirle un resultado final.

2.2. Imperativo VS Declarativo

2.2.1. Descripción imperativa

- Esta descripción consiste en indicar de forma detallada cada uno de los pasos que hay que dar, la descripción sería de esta forma:
 - Pela cuatro patatas y córtalas en trozos pequeños.
 - Pon una sartén al fuego con abundante aceite de oliva.
 - Fríe las patatas a fuego medio durante 15 minutos y escúrrelas.
 - Bate cinco huevos en un bol grande y añade un pellizco de sal.
 - Mezcla las patatas fritas con los huevos batidos.
 - Vierte la mezcla en una sartén con unas gotas de aceite caliente.
 - Deja que se cocine por un lado durante 3 minutos, dale la vuelta con ayuda de un plato y cocinar 2 minutos más.
 - Sácala y sírvela.
- **Justificación:** Esto es imperativo porque estoy dando un algoritmo exacto con la secuencia de instrucciones que debe de seguir el programa. Estoy diciendo como tiene que hacer las cosas paso por paso.

2.2.2. Descripción declarativa

- Esta descripción consiste en describir únicamente el resultado final que queremos lograr sin explicar como llegar a él, la descripción sería de esta forma:
 - Quiero una tortilla de patatas redonda, jugosa por dentro, cuajada por fuera y servida caliente en un plato.
- **Justificación:** Esto es declarativo porque estoy especificando al programa el estado o los valores que espero obtener al final. No le estoy dando instrucciones detalladas de cocina ni le estoy explicando cómo cocinarlo; solo le digo que es lo que quiero y el programa se encarga de averiguar cómo hacerlo.